

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

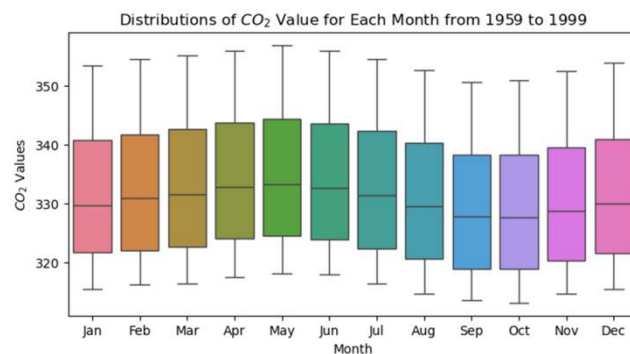
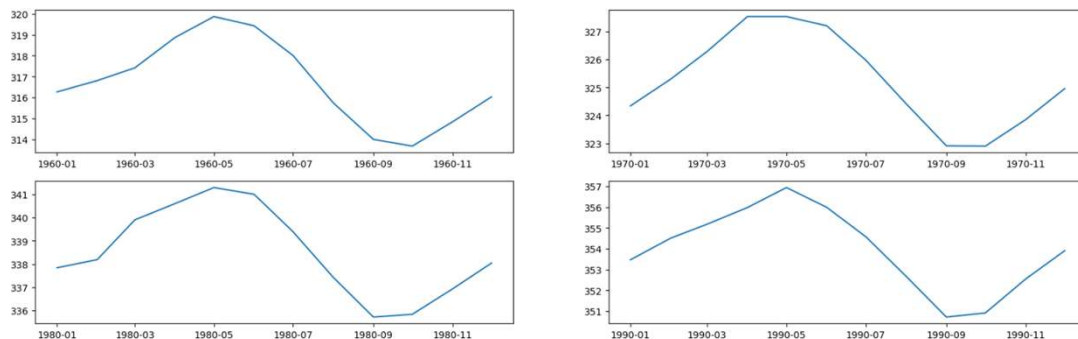
Kyungim Baek

Lecture 22

- Time Series Analysis (cont'd.)
 - Modeling the trend and seasonality

Previously... Modeling the Trend and Seasonality

1. Exploring the trend and seasonality
2. Forming a hypothesis about its components
3. Writing and testing the model to account for the trend and seasonality
 - Naturally, we cannot model the residual noise



Exploring the trend

- Here again, the process is exploratory.
- We can use any visualization that can help us identify the trend if one is present.

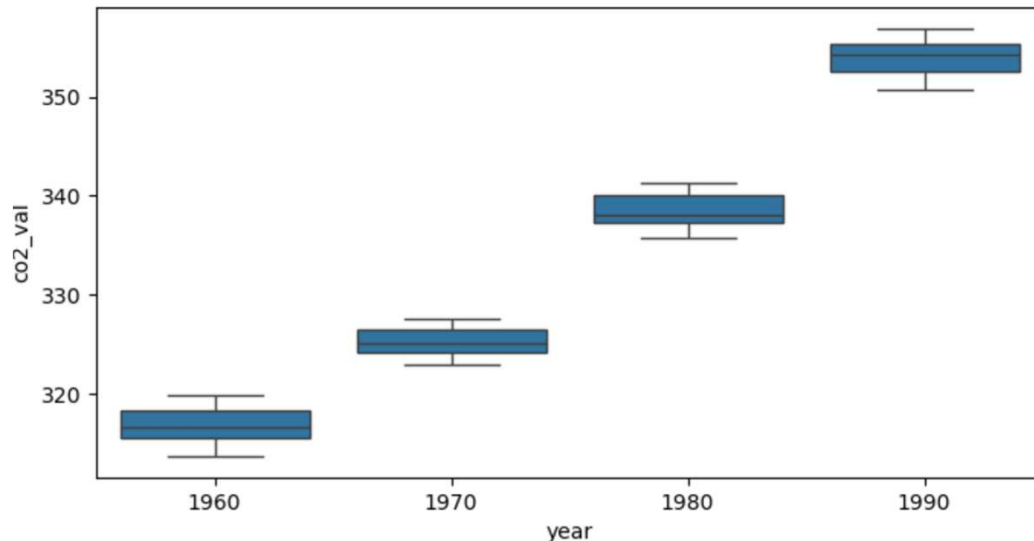
```
sample_4years = pd.DataFrame({"co2_val": sample_4years["co2_val"],  
                              'year': sample_4years.index.year})  
sample_4years.head()
```

	co2_val	year
1960-01-01	316.27	1960
1960-02-01	316.81	1960
1960-03-01	317.42	1960
1960-04-01	318.87	1960
1960-05-01	319.87	1960

original data

```
plt.figure(figsize=(8, 4))
```

```
sns.boxplot(x="year", y="co2_val", data=sample_4years);
```



Averaging / Aggregating data per period

- We can also inspect the overall trend by plotting the CO₂ values aggregated per year.
- We need to group the data by year.
 - Instead of using `groupby` as we did in the past, we will use `resample()` method.
 - `resample()` takes time related parameters that indicate how we intend to groupby.
 - E.g., YS means YEAR START.

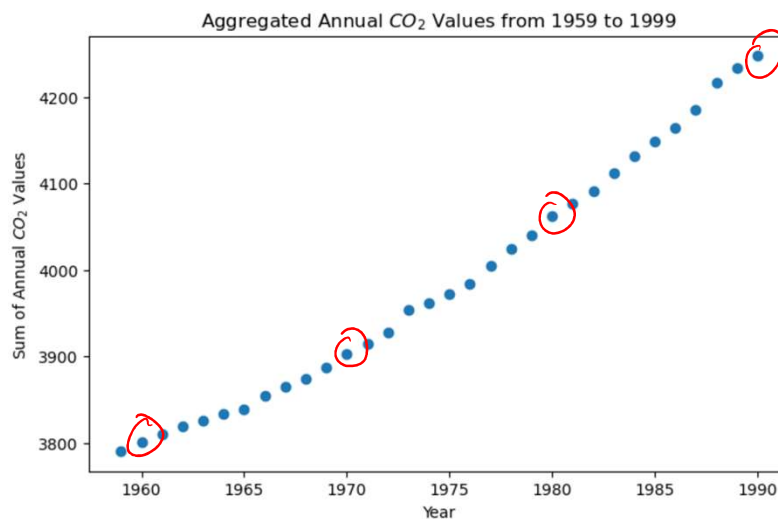
```
annual_sums = co2_data['co2_val'].resample('YS').sum()
annual_sums
```

monthly → yearly
384 → 32

1959-01-01	3789.920000	⋮	
1960-01-01	3800.980000	1976-01-01	3983.810000
1961-01-01	3809.830000	1977-01-01	4004.760000
1962-01-01	3819.560000	1978-01-01	4024.030000
1963-01-01	3825.990000	1979-01-01	4040.170000
1964-01-01	3833.549991	1980-01-01	4062.170000
1965-01-01	3838.470000	1981-01-01	4077.130000
1966-01-01	3854.540000	1982-01-01	4091.500000
1967-01-01	3864.240000	1983-01-01	4111.310000
1968-01-01	3873.980000	1984-01-01	4130.940000
1969-01-01	3887.190000	1985-01-01	4148.740000
1970-01-01	3903.240000	1986-01-01	4163.890000
1971-01-01	3913.880000	1987-01-01	4185.460000
1972-01-01	3927.540000	1988-01-01	4216.150000
1973-01-01	3954.130000	1989-01-01	4233.020000
1974-01-01	3960.980000	1990-01-01	4247.430000
1975-01-01	3971.810000		

Freq: YS-JAN, Name: co2_val, dtype: float64

```
plt.scatter(np.arange(len(annual_sums)) + 1959, annual_sums)
plt.title('Aggregated Annual $CO_2$ Values from 1959 to 1999')
plt.ylabel('Sum of Annual $CO_2$ Values')
plt.xlabel('Year')
```



2. Forming a hypothesis

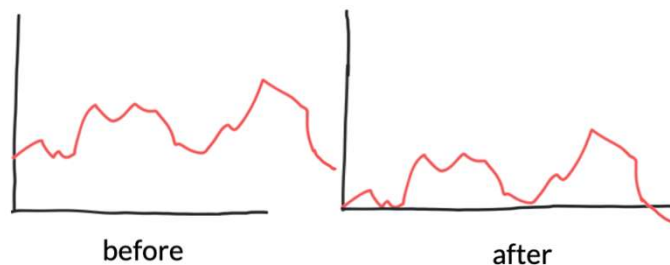
- Both visualizations show that there is a trend in the data.
 - A continuous increase in CO₂ concentrations over time.
 - Both are good ways to show the trend in the data.

12

Some useful time series modeling tips

It's usually helpful to:

1. Have the data pass through the origin.
 - Make it easier to tweak a function that represents the data.
 - This can be achieved by simply subtracting the offset from the data (shifting the time series).



Some useful time series modeling tips (cont'd.)

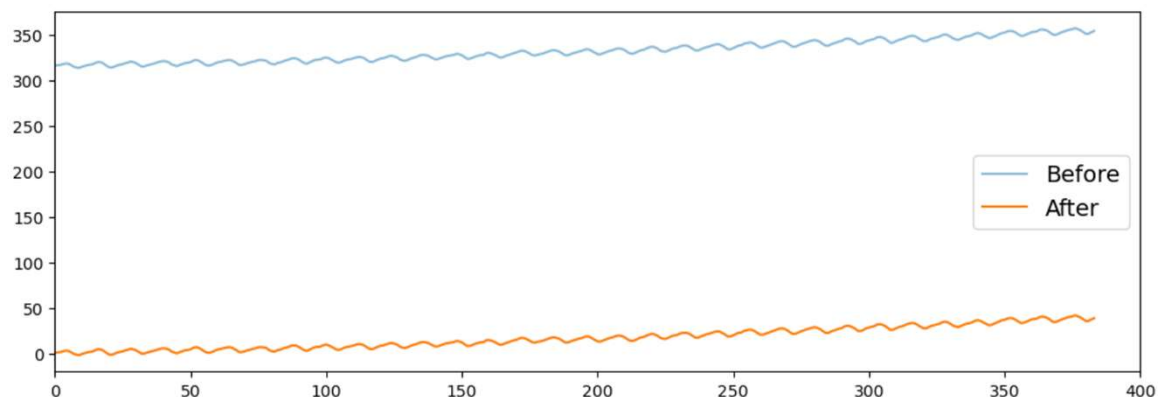
It's usually helpful to:

2. Represent the x-axis in a numerical form so that we can use it in the model.
 - Harder to model date or string labels.
 - For example, we can replace the year/month by a sequential number representing the number of months since the first measurement.

```
normalized_col = pd.DataFrame(co2_data['co2_val'] - co2_data['co2_val'].iloc[0])
normalized_col.index = range(0, len(normalized_col))

plt.figure(figsize=(12, 4))

plt.plot(normalized_col.index, co2_data['co2_val'], alpha=0.5, label="Before")
plt.plot(normalized_col['co2_val'], label="After")
plt.xlim(0, 400)
plt.legend(fontsize=14);
```



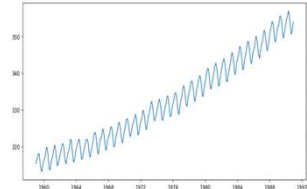
Using an appropriate model for the trend

Let's start modeling the data by describing the trend in the data.

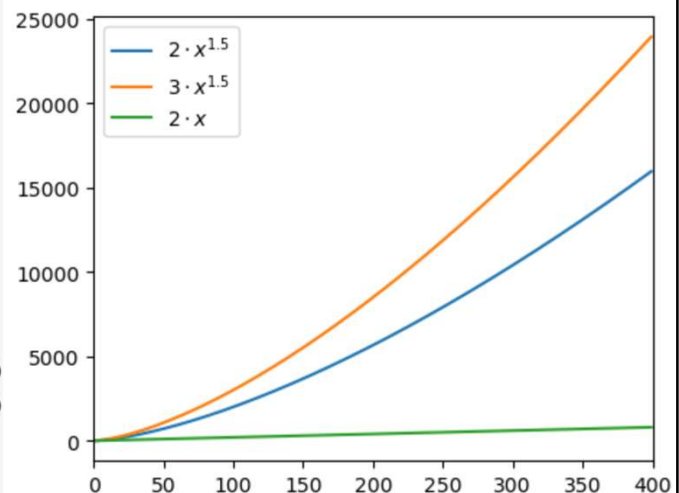
- How can we describe the trend?
 - The increase does not appear to be linear.
- Does it follow a power law?

$$f(x) = c_0 + c \cdot x^m$$

- Useful to be familiar with some of the most common parent functions.
 - Starting point that can be tweaked to provide better fits.



```
def func_powerlaw(x, m, c, c0):  
    return c0 + c * x**m  
  
x = np.arange(0, 400, 1)  
  
y_1 = func_powerlaw(x, 1.5, 2, 0)  
y_2 = func_powerlaw(x, 1.5, 3, 0)  
y_3 = func_powerlaw(x, 1., 2, 0)  
  
plt.figure(figsize=(5, 4))  
  
plt.plot(x, y_1, label="$2 \cdot x^{1.5}$")  
plt.plot(x, y_2, label="$3 \cdot x^{1.5}$")  
plt.plot(x, y_3, label="$2 \cdot x$")  
plt.xlim(0, 400)  
plt.legend();
```



What is a time series noise?

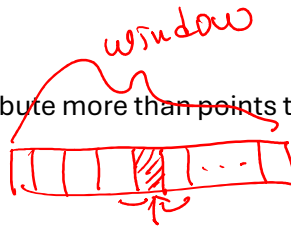
- Noise to be is an undesired additive signal with:
 - Constant mean
 - Constant standard deviation
 - Noise values at each time point are independent of each other

Working with noisy data

- Noise is often an inevitable part of time series.
- If we are only interested in trend, seasonality can be a form of noise.
- How can we correct for noise (here seasonality)?
 - Averaging or taking the sum of values per year as we did earlier?
 - We would lose the bulk of the data.
 - Started with monthly but we would end up with yearly.
 - Yields fewer points to use in training the model.

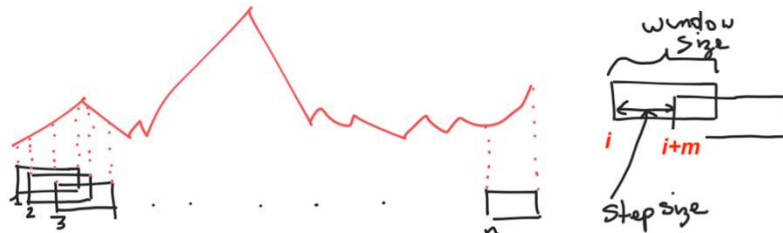
Smoothing with a sliding window

- Smoothing is used to capture essential patterns in the data by reducing noise.
 - Correct data so that points in a specific area are recomputed using some given function. For example:
 - as a mean of all the points in that area
 - Sliding windows (a.k.a, rectangular boxcar, unweighted sliding-average smooth, etc.).
 - weighted sliding window
 - E.g., points that are nearby will contribute more than points that are farther away.



Smoothing with a sliding window (cont'd.)

- When using a sliding window, a point's new value is computed using all the points in the window.
- Note that this is different from what we did earlier by averaging (or summing each year).
 - When averaging by year, 3-years worth of data is averaged into 3 data points.
 - With a sliding window of size 12, 3-years worth of data generates 25 data points.



```

x_axis = normalized_col.index
y_axis = normalized_col['co2_val'].rolling(window=12, center=True).mean()

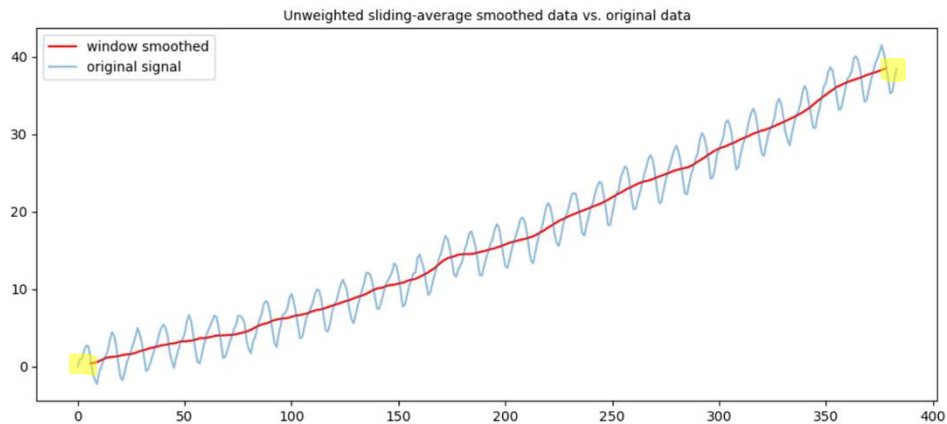
plt.figure(figsize=(12, 5))
plt.plot(x_axis, y_axis, 'r', label="window smoothed")
plt.plot(x_axis, normalized_col['co2_val'], alpha=0.5, label="original signal")
plt.title("Unweighted sliding-average smoothed data vs. original data", fontsize=10)
plt.legend();

```

False (circled in red)

6 (under '6' in 'window=12')

5 (under '5' in 'figsize=(12, 5)')



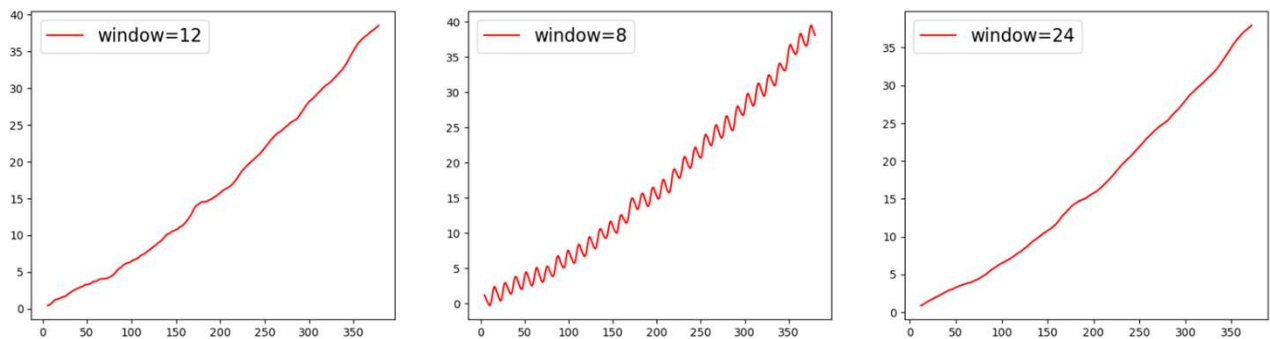
Question

- In the previous slide, we took a sliding window of size 12. What will happen if we change the size of the sliding window?
 - What happens if we take a larger sliding window?
 - What happens if we take a smaller sliding window?

```
y_axis_8 = normalized_col['co2_val'].rolling(window=8, center=True).mean()
y_axis_24 = normalized_col['co2_val'].rolling(window=24, center=True).mean()

plt.figure(figsize=(20, 5))

plt.subplot(1, 3, 1)
plt.plot(x_axis, y_axis, 'r', label="window=12")
plt.legend(fontsize=16)
plt.subplot(1, 3, 2)
plt.plot(x_axis, y_axis_8, 'r', label="window=8")
plt.legend(fontsize=16)
plt.subplot(1, 3, 3)
plt.plot(x_axis, y_axis_24, 'r', label="window=24")
plt.legend(fontsize=16);
```



```

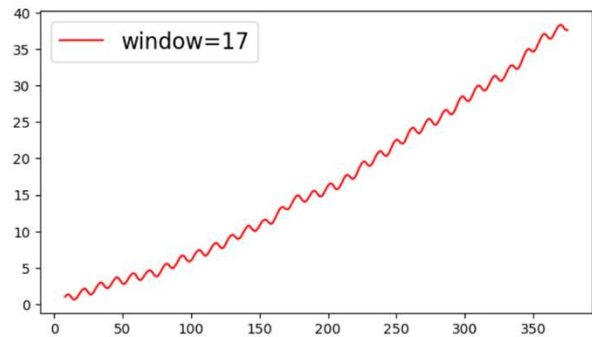
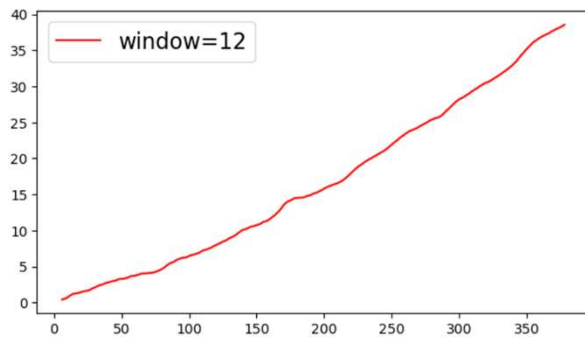
y_axis_17 = normalized_col['co2_val'].rolling(window=17, center=True).mean()

plt.figure(figsize=(20, 5))

plt.subplot(1, 2, 1)
plt.plot(x_axis, y_axis, 'r', label="window=12")
plt.subplot(1, 2, 2)
plt.plot(x_axis, y_axis_17, 'r', label="window=17")
plt.legend(fontsize=16);

```

Handwritten notes: *window = 13* (green), *12* (blue) with arrows pointing to the window size in the code.



Smoothing with a sliding window (cont'd.)

- Note that, with a window of size 12, the first 6 and last 5 observations are missing since we don't have sufficient data at these edges to average them.
- We can now model the trend that we observed in the smoother signal.
- We thought a power law could be a good approximation for the data.
 - Typically, you may (most likely will) have to try different functions to see which one fits the best.
 - For example, can we use an exponential function instead?

$$f(x) = c_0 + c \cdot m^x$$

cf. power-law: $f(x) = c_0 + c \cdot x^m$

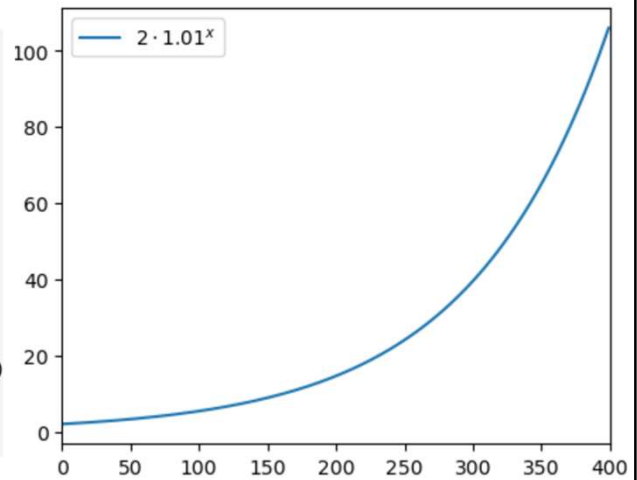
```
def func_exponential(x, m, c, c0):
    return c0 + c * (m**x)

x = np.arange(0, 400, 1)

y_1 = func_exponential(x, 1.01, 2, 0)

plt.figure(figsize=(5, 4))

plt.plot(x, y_1, label="$2 \\cdot {1.01}^x$")
plt.xlim(0, 400)
plt.legend();
```



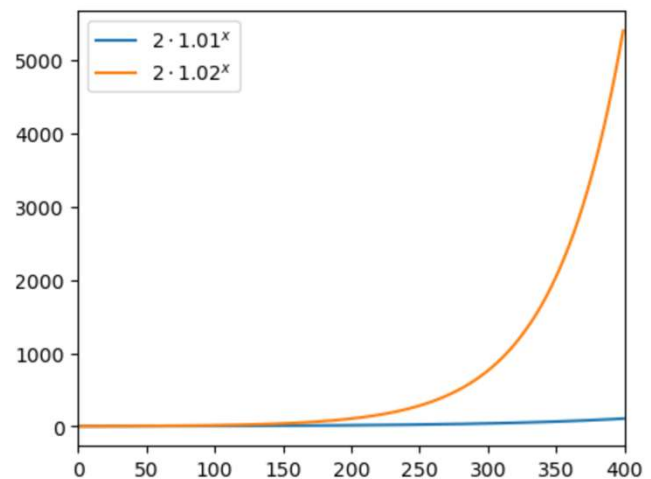
```
def func_exponential(x, m, c, c0):
    return c0 + c * (m**x)

x = np.arange(0, 400, 1)

y_1 = func_exponential(x, 1.01, 2, 0)
y_2 = func_exponential(x, 1.02, 2, 0)

plt.figure(figsize=(5, 4))

plt.plot(x, y_1, label="$2 \\cdot {1.01}^x$")
plt.plot(x, y_2, label="$2 \\cdot {1.02}^x$")
plt.xlim(0, 400)
```



Selecting the appropriate model

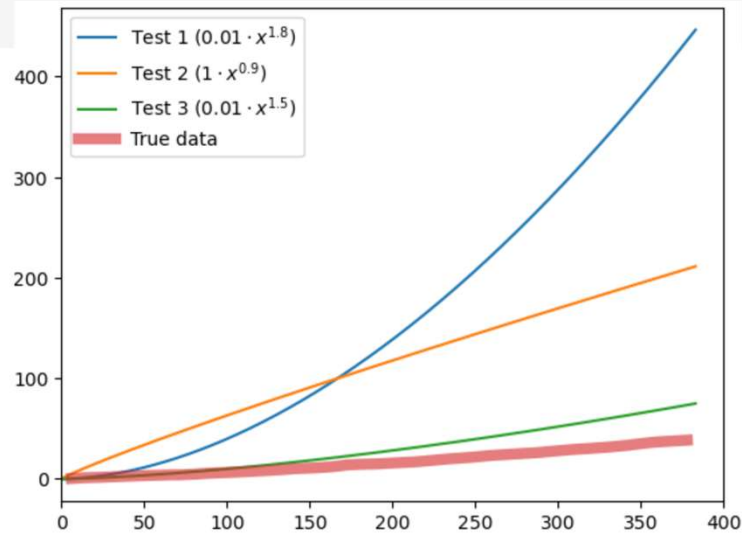
- Here, even if the exponential has the suitable shape, the parameters (e.g., m) will be difficult to find.
 - Minor perturbations will lead to major changes in the curve.
 - Minor perturbations will lead to poor fit.
- Power law is more appropriate here.

$$f(x) = c_0 + c \cdot x^m$$

Modeling the smoothed data

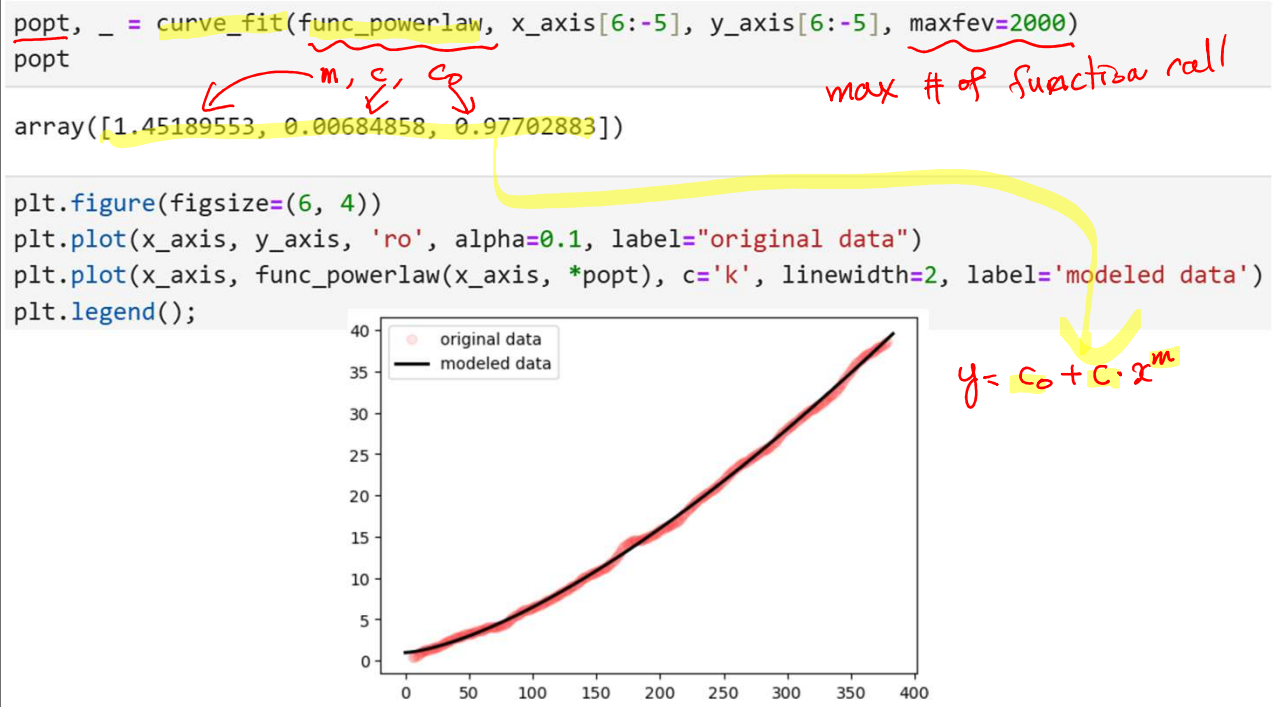
- We could manually tweak the parameters m , c , and c_0 to produce the best fit between our smoothed data and the power law function.
 - The closer the function is to the data, the better the fit.

```
plt.plot(x_axis, func_powerlaw(x_axis, 1.8, 0.01, 0), label='Test 1 ($0.01 \cdot x^{1.8}$'))
plt.plot(x_axis, func_powerlaw(x_axis, 0.9, 1, 0), label='Test 2 ($1 \cdot x^{0.9}$)')
plt.plot(x_axis, func_powerlaw(x_axis, 1.5, 0.01, 0), label='Test 3 ($0.01 \cdot x^{1.5}$)')
plt.plot(x_axis, y_axis, linewidth=6, alpha=0.6, label="True data")
plt.xlim(0, 400)
plt.legend();
```



Modeling the smoothed data (cont'd.)

- We can use the `curve_fit` function in `scipy.optimize` to identify the best values for the parameters (c_0 , c , and m).
 - https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html
- `curve_fit()`
 - Takes the data and the parameters we would like to optimize.
 - Returns:
 - `popt`: optimal parameters found by the curve fit
 - `pcov`: estimated covariance of `popt` which tells us how good is the fit against our data
 - We will ignore `pcov` and visually validate the fit instead.



Subtracting the trend

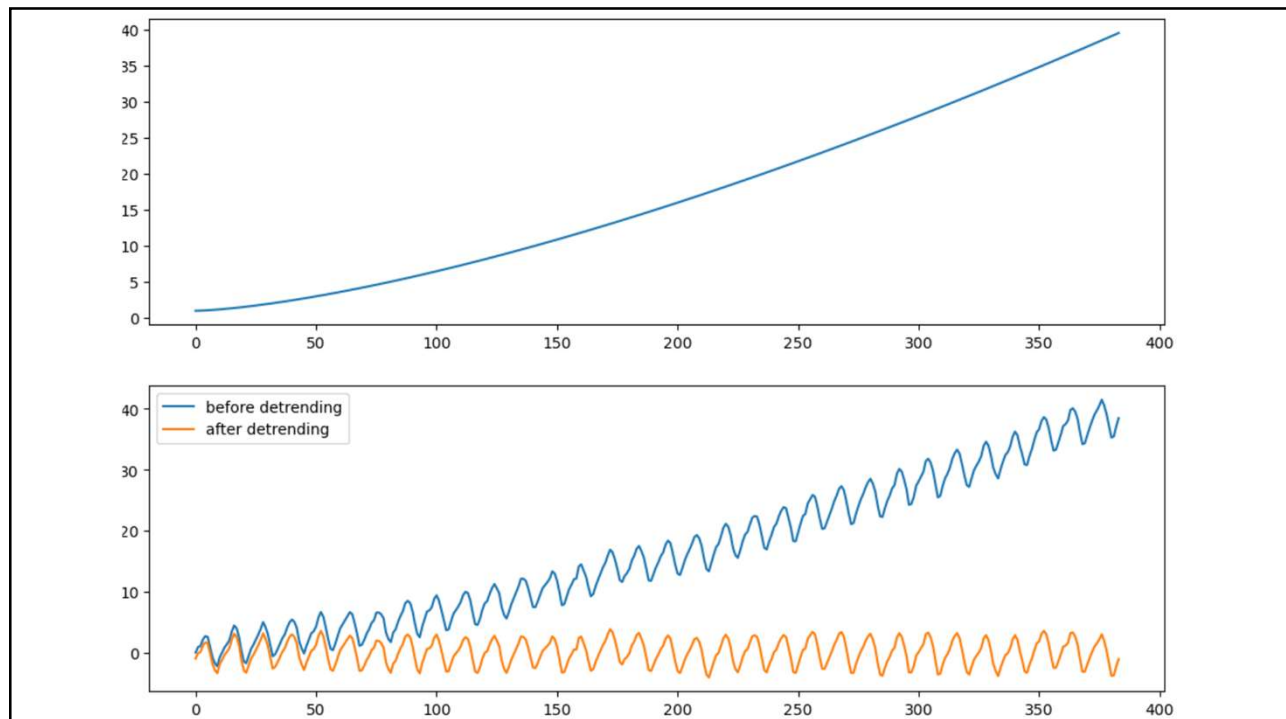
- We need to model the seasonal effect in the data.
- To do so, it's common to remove the trend in the data.
 - Similar to how we subtracted the seasonality using the smoothing when we were modeling the trend.
 - Allows us to focus on the seasonality while abstracting out the trend.
- Since we know the equation for the trend, we can use it to remove the trend at each time point.

```
f, ax = plt.subplots(2, figsize=(12, 8))

ax[0].plot(x_axis, func_powerlaw(x_axis, *popt))
ax[1].plot(x_axis, normalized_col['co2_val'], label="before detrending")

detrended = normalized_col['co2_val'] - func_powerlaw(x_axis, *popt)
ax[1].plot(x_axis, detrended, label="after detrending")
plt.legend();
```

↓
trend



Subtracting the trend (cont'd.)

- After removing the trend, we are now left with periodicity reflective of the seasonality that is in the data.
 - We don't need to worry about the trend when modeling the seasonality.
- The residual signal is periodic.

Modeling the periodic signal

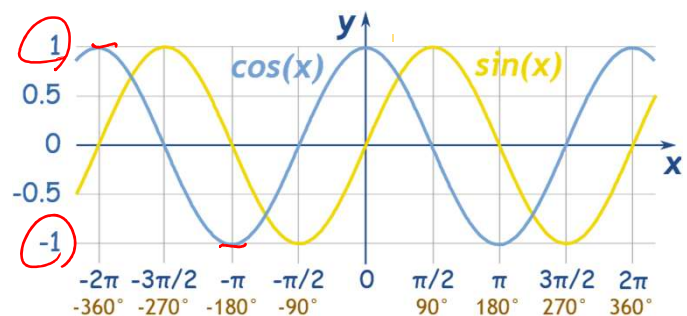
$$\text{period} = \frac{2\pi}{\text{freq.}} = \frac{2\pi}{|b|}$$

- Therefore, we can try to model the periodicity with a sine curve:

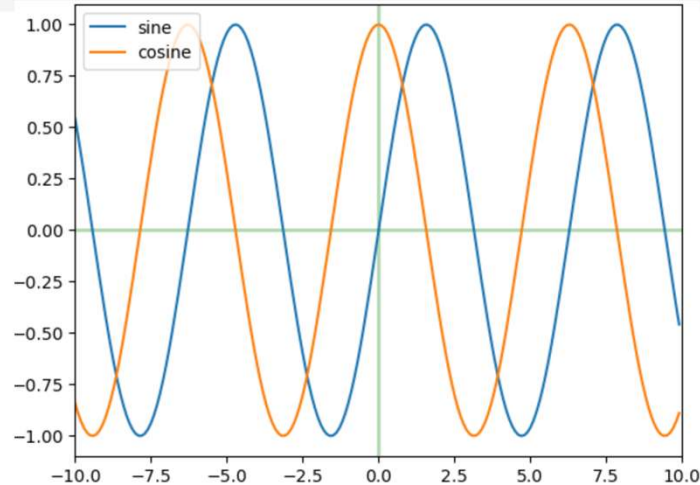
$$y = a \cdot \sin(b \cdot x)$$

$$y = 1 \cdot \sin x$$

- a controls the **amplitude**, or the height of the oscillations.
- b controls the **frequency**, and therefore the period of the cycle.
 - The natural period of the sine curve is 2π .



```
plt.axhline(y=0, linewidth=2, color='g', alpha=.3)
plt.axvline(x=0, linewidth=2, color='g', alpha=.3)
plt.xlim(-10, 10)
plt.plot(np.arange(-10, 10, 0.1), np.sin(np.arange(-10, 10, 0.1)), label="sine")
plt.plot(np.arange(-10, 10, 0.1), np.cos(np.arange(-10, 10, 0.1)), label="cosine")
plt.legend();
```



Modeling the periodic signal (cont'd.)

- For the period, recall that the data is atmospheric. (The cycle should repeat every 12 months?)
 - $2\pi = 12$ months
 - We simply divide by 12 so that for $x = 12, 24, 36, \dots$, we have

$$\sin\left(\frac{2\pi x}{12}\right): \sin\left(\frac{2\pi 12}{12}\right) = \sin\left(\frac{2\pi 24}{12}\right) = \sin\left(\frac{2\pi 36}{12}\right) = \dots = 0$$

$$\text{period} = \frac{2\pi}{|b|} = 12 \text{ months}$$

$$\Rightarrow |b| = \frac{2\pi}{12}$$

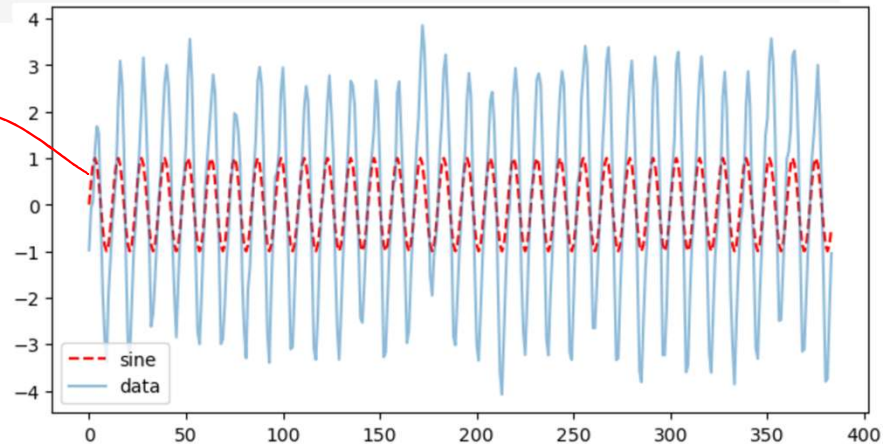
```
plt.figure(figsize=(8, 4))
```

```
plt.plot(x_axis, np.sin(2 * np.pi * x_axis / 12 ), 'r--', label='sine')
```

```
plt.plot(x_axis, detrended, label="data", alpha=0.5)
```

```
plt.legend();
```

$$\sin\left(\frac{2\pi}{12}x\right)$$



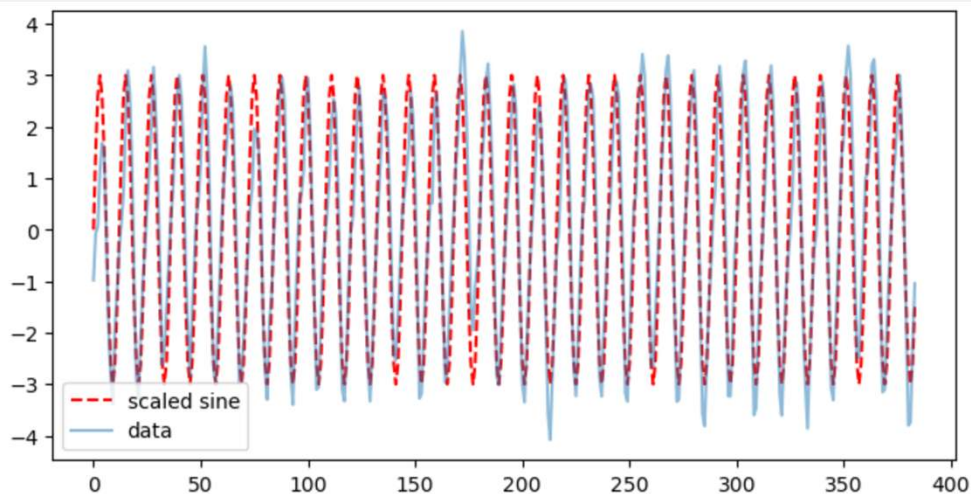
- For the amplitude, the data suggests a value of ~ 3 .
- We amplify (multiply) the signal by 3.

```
plt.figure(figsize=(8, 4))
```

```
plt.plot(x_axis, 3 * np.sin(2 * np.pi * x_axis / 12 ), 'r--', label='scaled sine')
```

```
plt.plot(x_axis, detrended, label="data", alpha=0.5)
```

```
plt.legend();
```

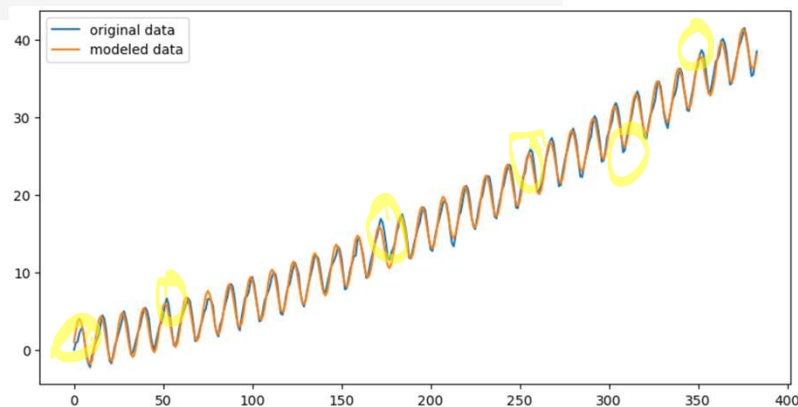


Modeling the data

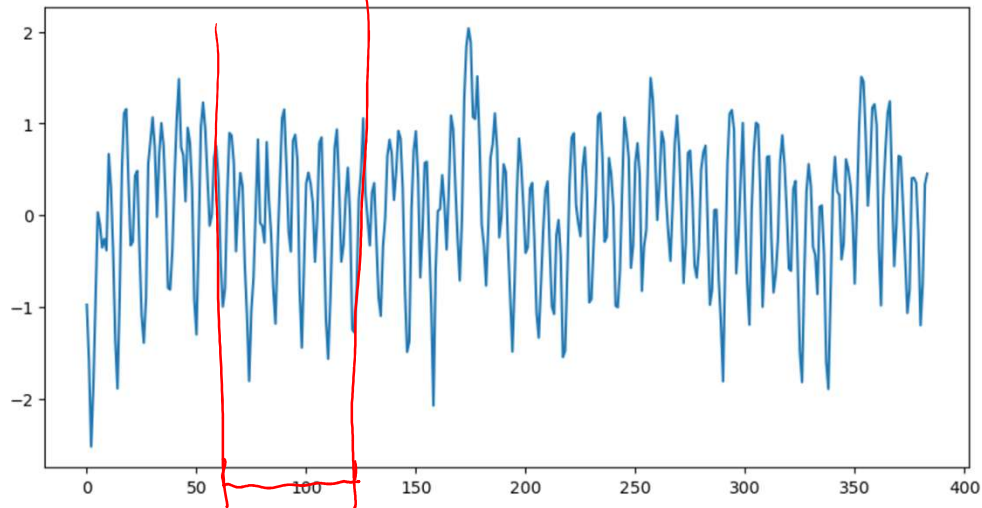
- As mentioned earlier, we can think of the time series y_t as a combination of three components:
 - A trend component $\leftarrow y_{\text{trend}} = C_0 + C \cdot x^n$
 - A seasonal component $\leftarrow y_{\text{season}} = 3 \cdot \sin\left(\frac{2\pi}{12}x\right)$
 - Noise, which usually are stochastic fluctuations, or errors
- Here, we will skip the error component and combine the trend and the seasonality to approximate the data.

```
def fx(x):  
    trend = func_powerlaw(x, *popt)  
    seasonality = (3 * np.sin(2 * np.pi * x / 12))  
    return trend + seasonality
```

```
plt.figure(figsize=(12, 6))  
plt.plot(normalized_col['co2_val'], label="original data")  
plt.plot(x_axis, fx(x_axis), label="modeled data")  
plt.legend();
```



```
plt.figure(figsize=(10, 5))  
residuals = normalized_col['co2_val'] - fx(x_axis)  
plt.plot(x_axis, residuals);
```



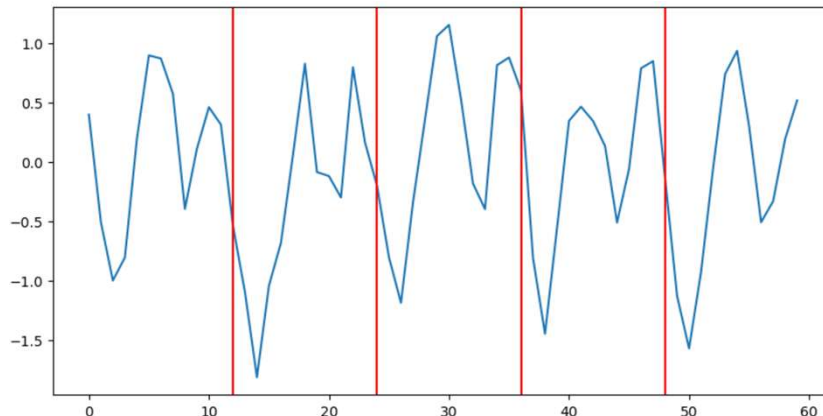
Residual seasonality

- The residuals are rather small here: avg $\sim [-2, 2]$.
- This fit is still not perfect as there is still some residual seasonality.
 - It's not as obvious as in the raw signal.
 - We can zoom on the residuals between regions $[60 \sim 120]$ to observe the remaining seasonality.


```
plt.figure(figsize=(10, 5))

plt.plot(range(len(residuals[60: 120])), residuals[60: 120])

plt.axvline(x=12, color='r')
plt.axvline(x=24, color='r')
plt.axvline(x=36, color='r')
plt.axvline(x=48, color='r');
```



```
print(residuals.sum())
residuals.head()
```

11.326281299185487

```
0    -0.977029
1    -1.583877
2    -2.523841
3    -1.870783
4    -0.916359
dtype: float64
```

Residual seasonality (cont'd.)

- We note that there is still repeating structure occurring approximately every 12 months.
 - [low, high, low, high, low], [low, high, low, high, low], ...
- This is since our sine wave does not fully capture all the seasonality in the data.
 - We need to consider higher harmonics.
- A **harmonic** is simply a sine wave with frequency f_n such that:

$$f_n = n \times f_1$$

where n is a positive integer.

- I.e., f_n is an integer multiple of the fundamental frequency f_1 (first harmonic).

→ fundamental freq.


```
plt.figure(figsize=(20, 8))

x = np.arange(0, 60, 0.05)

plt.plot(x, 2 * np.sin(x), 'k-', label='2 sine(x)')
plt.plot(x, 1 * np.sin(2 * x), 'r-', label='1 sine(2*x)')
plt.plot(x, 0.5 * np.sin(4 * x), 'b-', label='0.5 (sine4*x)')
plt.axhline(y=0, linewidth=2, color='g', alpha=.3)
plt.legend(fontsize=16);
```

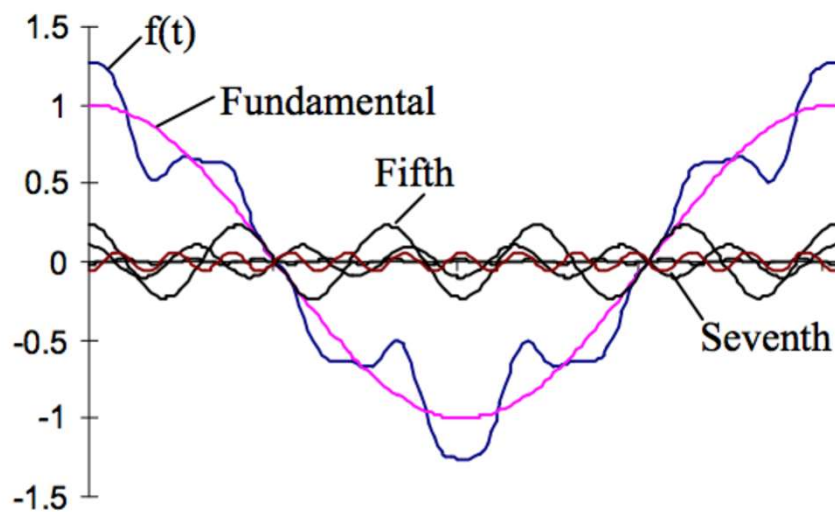
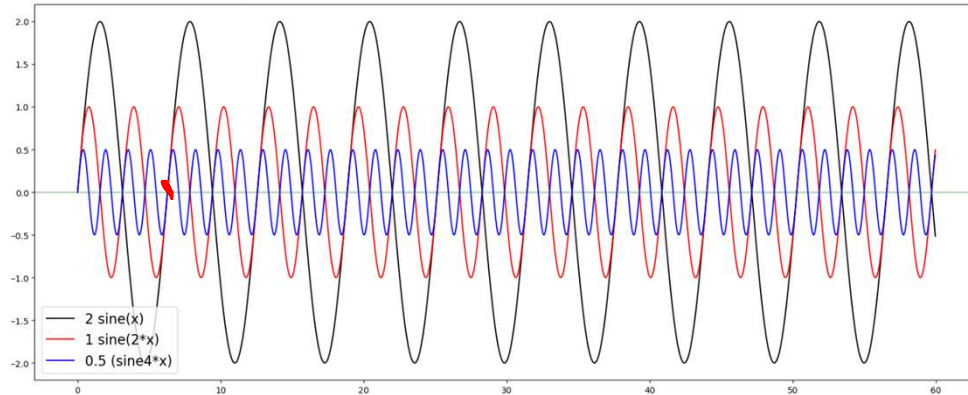


Figure from Ranade and Xu, An Overview of Harmonics Modeling and Simulation.

Residual seasonality (cont'd.)

- Therefore, to obtain the second harmonic, we can divide the period by 2 (period $\sim 1/\text{frequency}$).

```
seasonality = (3 * np.sin(2 * np.pi * x / 12)) +  
              (0.75 * np.sin(2 * np.pi * x / 6))
```

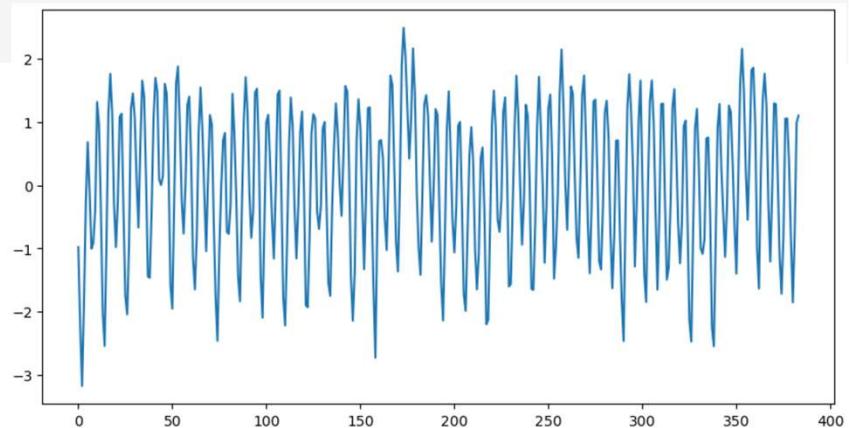
$\rightarrow 3 \cdot \sin\left(\frac{2\pi}{12}x\right)$

$\hookrightarrow \underline{0.75} \sin\left(\frac{2\pi}{6}x\right)$

```
def fx(x):  
    trend = func_powerlaw(x, *popt) 1st  
    seasonality = (3 * np.sin(2 * np.pi * x / 12)) + (0.75 * np.sin(2 * np.pi * x / 6)) 2nd harmonic  
    return trend + seasonality
```

```
residuals = normalized_col['co2_val'] - fx(x_axis)
```

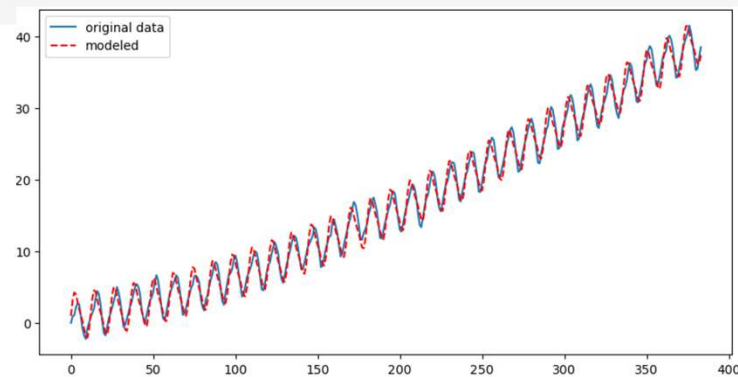
```
plt.figure(figsize=(10, 5))  
plt.plot(residuals);
```



Final model

- Plotting the overall model.
 - Power law trend to capture the long-term increase
 - 12-month seasonality to model the yearly cycle
 - Second harmonic (6-month cycle) to refine and better capture the seasonal fluctuation
- We can observe that our model fits the data rather well.

```
def fx(x):  
    trend = func_powerlaw(x, *popt)  
    seasonality = (3 * np.sin(2 * np.pi * x / 12)) + (0.75 * np.sin(2 * np.pi * x / 6))  
    return trend + seasonality  
  
ax = plt.figure(figsize=(10, 5))  
  
plt.plot(normalized_col['co2_val'], label="original data")  
plt.plot(fx(x_axis), linestyle='--', color='r', label="modeled")  
plt.legend();
```



Predicting future data

- We predict 3 years (36 months) of CO₂ values.

```
plt.figure(figsize=(10, 5))

x_axis = np.arange(0, len(normalized_col['co2_val']) + 36)

plt.plot(normalized_col['co2_val'], label="original data")
plt.plot(fx(x_axis), linestyle='--', color='r', label="modeled")
plt.legend();
```

3 years

